Assignment "Correctness Proofs", B3STV 2025/26

Deadlines: see website.

Part I (3pt)

1. **Refreshing Predicate Logic** [1pt]. Consider the formula below (it is the same as in exercise no. 7 from the Lecture Notes Section 3.9):

   $$\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}]) \quad \wedge \quad (\forall \mathtt{i} : \mathtt{j} \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}])$$
   $$\Rightarrow$$
   $$\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{j} : \mathtt{b}[\mathtt{i}])$$

   Notice that all occurences of `i` are bound, whereas `j` is free.

   Below is an attemp for a formal proof that the above formula is valid. At the top level the proof's idea is to use a proof-by-contradiction. So, rather than proving G directly, it proceeds by first trying to prove $\neg G \Rightarrow$ **false**. Few steps further: it plans to finish the proof by performing a case-split argumentation. Unfortunately the author went awol, and the proof is left unfinished.

   Your task is to finish the proof using the said case-split (see the hint in the proof body).

   - Keep to the same proof style (as in the Lecture Notes).
   - Steps involving quantifiers ($\forall$ and $\exists$) are to be worked out in small steps.

   PROOF Top
   [A1:] $\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}])$
   [A2:] $(\forall \mathtt{i} : \mathtt{j} \leq \mathtt{i} < \mathtt{n} : \mathtt{b}[\mathtt{i}])$
   [G:] $\neg(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{j} : \mathtt{b}[\mathtt{i}])$
   BEGIN
   1. { see the subproof below } $(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{j} : \mathtt{b}[\mathtt{i}]) \Rightarrow$ **false**

      PROOF subp
      [A1:] $(\forall \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{j} : \mathtt{b}[\mathtt{i}])$
      [G:] **false**
      BEGIN
      1. { rewriting Top.A1 using Negate-$\forall$ (Theorem A.4.7) }
      $(\exists \mathtt{i} : 0 \leq \mathtt{i} < \mathtt{n} : \neg \mathtt{b}[\mathtt{i}])$

      2. { Applying $\exists$-elimination on 1 } [SOME i] $0 \leq \mathtt{i} < \mathtt{n} \wedge \neg \mathtt{b}[\mathtt{i}]$

      ... finish the proof by doing a case split on $\mathtt{j} \leq \mathtt{i} \vee \mathtt{j} > \mathtt{i}$.
      ...
      ...
      END

   ...
   ... well, you are not quite done yet: you don't have G yet. Give your finishing step/s here
   END

2. **Induction and Equational Proof** [1.2pt] Consider the following two functions to calculate the sum of the elements in a list/sequence. In the notation below, [ ] denotes an empty list, and $x{:}s$ denotes a list with x as its first element and s as the remaining of the list.

   From programming perspective, these functions can be seen as *programs* that get a list as input and return an integer as output. In fact, you can write them as programs in a functional programming language like Haskell. In programming, the above mentioned [ ] and : act as list constructors.

   $$\begin{array}{lcl} \mathtt{sum1}\ [\,] & = & 0 \\ \mathtt{sum1}\ (\mathtt{x} : \mathtt{s}) & = & \mathtt{x} + \mathtt{sum1}\ \mathtt{s} \end{array}$$

   $$\begin{array}{lcl} \mathtt{sum2}\ \mathtt{n}\ [\,] & = & \mathtt{n} \\ \mathtt{sum2}\ \mathtt{n}\ (\mathtt{x} : \mathtt{s}) & = & \mathtt{sum2}\ (\mathtt{x} + \mathtt{n})\ \mathtt{s} \end{array}$$

We want to prove that we have:

$$\texttt{sum1 s} \;=\; \texttt{sum2 0 s} \quad \text{, for all } \texttt{s} \tag{1}$$

If this is true, then calls to `sum1` can be replaced by `sum2 0`, if we so desire.

To prove a general property about lists, such as the one above, you typically need to use the list induction rule (quite analogous to the natural number induction rule that you already know):

$$\frac{P\,[\,] \quad , \quad (\forall x :: (\forall s :: P\ s \Rightarrow P\ (x{:}s)))}{(\forall s :: P\ s)}$$

To prove the equation in (1), $P\ s$ would be the formula "`sum1 s = sum2 0 s`". Unfortunately in this particular case applying the rule directly like that does not work. You are encouraged to try, and discuss with fellow students or with a TA, how and why the proof gets stuck.

Instead we will now prove a more general statement:

$$(\forall \texttt{n} :: \texttt{n} + \texttt{sum1 s} \;=\; \texttt{sum2 n s}) \quad \text{, for all } \texttt{s} \tag{2}$$

Quite obviously this implies the original equality (1) that you wanted to prove. **Your task** is to formally prove the above generalized statement (2). And yes, using induction.

As before, use the same proof style as in the Lecture Notes.

As a side note, `sum2` has the overhead of having to maintain one extra parameter, but it can be optimized to a more efficient program because it does not actually need to build a call stack.

3. **Weakest pre-condition** [0.5pt]. What is the weakest pre-condition (**wp**) of the following statements with respect to the given post-conditions. All variables are of type `int`.

   Do not just guess the **wp**. Use the **wp**-rules you learned from the lectures to calculate it.

   (a) $\texttt{x} := \texttt{x+1};\ \texttt{y} := \texttt{x} * \texttt{y} \quad \{* \ \texttt{x} > 0 \wedge \texttt{y} > \texttt{z} \ *\}$

   (b) **if** $\texttt{x>y}$
   then $\texttt{x} := \texttt{x-y}$
   else $\texttt{y} := \texttt{y-x}$
   $\{* \ (\exists \texttt{k} : \texttt{k} > 0 : \texttt{d*k} = \texttt{x}) \wedge (\exists \texttt{n} : \texttt{n} > 0 : \texttt{d*n} = \texttt{y}) \ *\}$

4. **Correctness of simple statement** [0.3pt]. Here is again the statement from 2b, this time with a full specification. All variables are of type `int`. To give you a bit intuition: the formula $(\exists \texttt{k} : \texttt{k} > 0 : \texttt{d*k} = \texttt{x})$ says that `d` is thus a divisor of `x`.

   $\{* \ \texttt{d>0} \wedge \texttt{x} \neq \texttt{y} \wedge (\exists \texttt{k} : \texttt{k} > 0 : \texttt{d*k} = \texttt{x}) \wedge (\exists \texttt{n} : \texttt{n} > 0 : \texttt{d*n} = \texttt{y}) \ *\}$

   if $\texttt{x>y}$
   then $\texttt{x} := \texttt{x-y}$
   else $\texttt{y} := \texttt{y-x}$

   $\{* \ (\exists \texttt{k} : \texttt{k} > 0 : \texttt{d*k} = \texttt{x}) \wedge (\exists \texttt{n} : \texttt{n} > 0 : \texttt{d*n} = \texttt{y}) \ *\}$

   Suppose we want to formally prove that the above specification is satisfied. Questions:

   (a) How to use what you learned from 3b?

   (b) Give the *header* of your formal proof. That is, list your assumptions and goal(s). You only need to give the header, not a full proof.

Part II (7pt + 0.5)

For some of the assignments in this section you can use the tool `stv−checker` to help you checking your candidate invariant and termination metric. You don't have to use it, and as a disclaimer: it is an exeperimental tool. The tool uses testing to validate your invariant and metric. The tool is just to assist you; it does *not* do grading. It does not produce a proof either, mind you. And you need to read its README first, if you want to use it.

1. **Invariant** [1pt; 0.5pt each]. Give an invariant for each of the programs below, that would be good enough to prove their corresponding specifications. You do not need to deliver any proof, but do check it on your own scratch that your proposed invariants are indeed good.

   (a)
   $\{* \; i{=}2 \;\wedge\; j{=}20 \; *\}$
   `while` $i{<}j$ `do` $\{ \; j{:=}j{-}2 \; ; \; i{:=}i{+}1 \; \}$
   $\{* \; j = i \; *\}$

   The needed invariant is less trivial than you might thought, e.g. just $\mathtt{i} \leq \mathtt{j}$ as I is not good enough, while intuitively you might expect it to be an invariant. That intuition is not totally off. However, if you consider the proof obligation IC:

   $$\{* \; \underbrace{\mathtt{i{<}j}}_{g} \wedge \mathtt{i{\leq}j} \; *\} \; \mathtt{j} := \mathtt{j{-}2} \; ; \; \mathtt{i} := \mathtt{i{+}1} \; \{* \; \mathtt{i{\leq}j} \; *\}$$

   the post-condition cannot be proven. Perhaps the problem is that the candidate $\mathtt{i} \leq \mathtt{j}$ does not contain enough information to close the proof of the post-condition above, and you might want to consider extending it.

   If you want to use `stv−checker` to check your candidate invariant, see file `exercises_invs/P1.py`.

   (b) Below is an algorithm to efficiently multiply two integers. It is a very old algorithm, written by an Egyptian A'h-mose, back in 1700 BC! It is the same algorithm used in CPU. The $*2$ and `div` 2 operations used in the algorithm can be done efficiently by, essentially, shifting the bit strings representing the target integer.

   ```
   {* x = X0 ∧ y = Y0 ∧ x > 0 ∧ y > 0 *} — pre−condition
    MUL(x,y:int) : int
    { var z ;
      z:=0 ;
      while x ≠ 0 do
            if odd x then { z:=z+y ; x:=x−1 }
                     else { y:=y*2 ; x := x div 2 } ;
      {* z = X0 * Y0 *} — post−condition
      return z
    }
   ```

   If you want to use `stv−checker` to check your candidate invariant, see file `exercises_invs/MUL.py`.

2. **Termination metric** [0.5pt]. Consider the following program. It does not do anything exciting, but it still makes an interesting case for you. Give an invariant and a termination metric that would be good enough to prove that the program terminates with the specified post-condition. You do not need to deliver any proof.

   $\{* \; k = 1 \; *\}$
   `while` $k{\neq}10$ `do`
   `if` $k{=}1$ `then` $k{:=}0$ `else` $k{:=}k{+}2$
   $\{* \; k = 10 \; *\}$

   Note that a simple $10 - k$ won't work as your termination metric, as this expression does not always decrease during your iterations.

   If you want to use `stv−checker` to check your candidate invariant and termination metric, see file `exercises_invs/PT1.py`.

3. **Proving correctness of a loop** [3.5pt]. The program below checks if `x` occurs as an element of the array-segment `a[0..N)`. The part of the pre-condition that says that the array is sorted is only relevant for later in question No. 5. For now you can ignore that part.

```
{* N≥0 ∧ sorted a N *}

 found := false ;
 i := 0 ;
 while i < N do { found := found ∨ (a[i]=x) ; i:=i+1 }

{* ?? *}
```

The predicate sorted expresses that the array-segment `a[0..N)` is ascendingly sorted:

$$\texttt{sorted a N} \;=\; (\forall \texttt{i} : 0 \le \texttt{i} < \texttt{N} : (\forall \texttt{j} : \texttt{i} \le \texttt{j} < \texttt{N} : \texttt{a}[\texttt{i}] \le \texttt{a}[\texttt{j}]))$$

Give a formal specification for the program's post-condition, and then give a formal proof of the correctness of the above program (under **total correctness**, so you have to prove its termination as well). You do not need to show the calculation of **wp**.

Additional requirements (also apply for questions No 4 and 5):

- Use the proof style as in the Lecture Notes.
- Clearly justify each proof step with a comment. Your justification is your argument why you consider the step to be a valid proof step; this is just as important as the step itself.
- Proof steps involving quantifications ($\exists$ and $\forall$) should be done in "small" steps (avoid merging such steps into a single one big proof step). A "small" step is defined as a step for which there is a rule or theorem justifying it listed in the Lecture Notes.

  For example, this fragment of an equational proof is good:

  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k}+1 : \texttt{b}[\texttt{i}])$
  $= \{$ we assumed that $0 \le k$, then using Domain Merging $\}$
  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k} \vee \texttt{i} = \texttt{k} : \texttt{b}[\texttt{i}])$
  $= \{ \forall$ Domain Split $\}$
  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k} : \texttt{b}[\texttt{i}]) \;\wedge\; (\forall \texttt{i} : \texttt{i} = \texttt{k} : \texttt{b}[\texttt{i}])$
  $= \{ \forall$ Quantification over Singleton Domain $\}$
  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k} : \texttt{b}[\texttt{i}]) \;\wedge\; \texttt{b}[\texttt{k}]$

  The fragment below, which merges the above steps into one, is **not** allowed in this assignment, since it makes it impossible for your evaluators to determine if you actually understand the underlying formal reasoning:

  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k}+1 : \texttt{b}[\texttt{i}])$
  $= \{$ we assumed $0 \le k$ $\}$
  $(\forall \texttt{i} : 0 \le \texttt{i} < \texttt{k} : \texttt{b}[\texttt{i}]) \;\wedge\; \texttt{b}[\texttt{k}]$

4. **A loop with array assignment** [2pt].

The following program traverses an array `a` from `a[0]` to `a[N-1]`, where `N` $\geq$ `1`. If it finds that at a position `i<N−1` the element `a[i]` is greater than `a[i+1]`, it swaps the two elements. This way, once the loop terminates, no element in `a` positioned at an index less than `N-1` is greater than `a[N-1]`:

```
{* N≥1 *}
i := 0;
while ¬ (i = N − 1) do {
  if a[i] > a[i + 1] then {
    tmp:=a[i]; a[i]:=a[i+1]; a[i+1]:=tmp
  }
  else skip;
  i := i+1;
}
{* (∀k : 0≤k<N−1 : a[k]≤a[N−1]) *}
```

Provide a formal proof to show that this is a valid specification with respect to a **partial correctness** interpretation. Check the Lecture Notes on how to handle assignments to arrays.

When calculating the wp of the then-part inside the loop, you can use the following lemma (you have to prove it first, if you use it):

Lemma: for $0 \leq i$ we have:
$\mathsf{wp}\ (a[i] := a[i+1]; a[i+1] := tmp)\ (\forall k : 0 \leq k < i{+}1 : a[k] \leq a[i+1])$
$=$
$(\forall k : 0 \leq k < i : a[k] \leq tmp)\ \wedge\ a[i+1] \leq tmp)$

Some tips. When trying to prove IC, you will have to calculate wp *body Inv*. Because the loop's body above involves an if-then-else and array assignments, calculating this wp is quite involved. Try to simplify the resulting formula; this helps later in the proof of IC. Few further tips that might help in this calculation:

- Use the following variant of the wp rule of if-then-else:

  $$\mathsf{wp}\ (\textbf{if}\ g\ \textbf{then}\ S\ \textbf{else}\ T)\ Q \quad = \quad (g \Rightarrow \mathsf{wp}\ S\ Q)\ \wedge\ (\neg g \Rightarrow \mathsf{wp}\ T\ Q)$$

- Section 6.9 of the LN mentions that wp is distributive over conjunction:

  $$\mathsf{wp}\ S\ (Q_1 \wedge Q_2)\ =\ (\mathsf{wp}\ S\ Q_1) \wedge (\mathsf{wp}\ S\ Q_2)$$

  You can use this to split the calculation of wp into several parts for beter overview.

- Calculating wp over an array assignment might generate sub-formulas of the form

  $$a(i\ \textbf{repby}\ e)[k]$$

  By the definition of **repby**, notice that this formula can be rewritten to an if-then-else expression

  $$(k = i) \rightarrow e \mid a[k]$$

  Apply this 'reduction'.

- There are theorems in Appendix A for rewriting if-then-else expressions. E.g. you can rewrite this:
  $(cond \rightarrow x \mid y) \leq 0$
  $=$ // Theorem A.4.2
  $cond \rightarrow\ x \leq 0 \mid y \leq 0$
  $=$ // Theorem A.4.5 (COND-split)
  $(cond \Rightarrow x \leq 0)\ \wedge\ (\neg cond \Rightarrow y \leq 0)$

If you want to use `stv–checker` to check your candidate invariant, see file `exercises_invs/BUBLE.py`.

5. **Optional (hard): Loop with breaks** [0.5 pt]. Here is a more efficient variation of the program in No. 3. The variant below will 'break' the loop right after `x` is found, or when the current `a[i]` exceeds `x` (well, `a` is sorted, so searching further would then be vain).

```
{* N≥0 ∧ sorted a N *}

 found := false ;
 i := 0 ;
 while i<N ∧ ¬found ∧ a[i]≤x do {
     found := found ∨ (a[i]=x) ;
     i:=i+1
 }

{*  same post-cond as in the simple variant in No. 3  *}
```

Formally prove the correctness of the above program. Since in No. 3 you already proved the correctness of the 'simple' variant that has no break, for the above program you only need to redo your PEC (Proof of the Exit Condition). The pre-condition says that the array is sorted, `sorted a N`. Since the program does not change the array `a`, in your PEC you can add this to your assumptions.

Additional Requirements: the same as in No. 3.